

Chapter 26 — Graph BFS

0. What You Will Learn in This Chapter

Graph BFS is one of the most important graph traversal patterns.

BFS means:

Breadth-First Search

Graph BFS explores nodes level by level from a starting node.

It is especially useful when you need the **shortest path in an unweighted graph**.

By the end of this chapter, you should understand:

- What Graph BFS is
- How Graph BFS differs from Graph DFS
- Why BFS uses a queue
- How BFS finds shortest path in unweighted graphs
- How to use visited sets
- How to process graph levels
- How to solve grid BFS problems
- How to solve multi-source BFS problems
- How to solve state-space BFS problems
- How to solve shortest transformation problems
- How to avoid infinite loops
- How to reason about BFS complexity

Core problems covered in this chapter:

1. Find if Path Exists in Graph
2. Shortest Path in Binary Matrix
3. Rotting Oranges
4. Walls and Gates
5. Open the Lock
6. Word Ladder
7. Minimum Genetic Mutation
8. Nearest Exit from Entrance in Maze
9. Snakes and Ladders
10. 01 Matrix
11. As Far from Land as Possible
12. Shortest Bridge
13. Bus Routes
14. All Nodes Distance K in Binary Tree
15. Minimum Knight Moves
16. Shortest Path with Alternating Colors
17. Shortest Path in a Grid with Obstacles Elimination
18. Shortest Path Visiting All Nodes

1. What Is Graph BFS?

Graph BFS is a traversal method that visits nodes in increasing distance from the start node.

Given graph:

```
0 -- 1 -- 3
|
2
```

BFS from 0 visits:

distance 0: 0
distance 1: 1, 2
distance 2: 3

So BFS explores:

near nodes first
far nodes later

This is why BFS is ideal for shortest path in unweighted graphs.

2. Why Graph BFS Exists

DFS goes deep first.

BFS goes wide first.

If every edge has equal cost, then the first time BFS reaches a node, it has found the shortest number of edges to that node.

Example:

```
0 -- 1 -- 3
 \   /
  2 ----
```

Shortest path from 0 to 3 is:

0 -> 2 -> 3

BFS finds it by exploring all distance-1 nodes before distance-2 nodes.

3. Graph BFS vs Graph DFS

Data structure	stack / recursion	queue
Traversal style	deep first	level first
Good for	components, reachability, cycles	shortest unweighted path
Finds shortest path?	no, not generally	yes, unweighted
Space	path depth	frontier width
Grid use	islands/regions	shortest

Feature	DFS	BFS
		distance/spread

DFS can tell whether a path exists.

BFS can tell the shortest number of steps.

4. Core Need / Why Use BFS?

Use BFS when the problem asks:

minimum number of steps
 minimum moves
 shortest path
 nearest target
 fewest transformations
 spread level by level

BFS naturally models:

- infection spread
 - fire spread
 - rotting oranges
 - word transformations
 - lock combinations
 - shortest maze path
 - nearest exit
 - distance to nearest zero
 - minimum jumps/moves
-

5. When to Use Graph BFS

Use Graph BFS when:

5.1 Shortest Path in Unweighted Graph

Examples:

- Shortest Path in Binary Matrix
- Word Ladder
- Open the Lock

5.2 Minimum Number of Moves

Examples:

- Snakes and Ladders
- Minimum Knight Moves
- Nearest Exit from Entrance in Maze

5.3 Multi-Source Spread

Examples:

- Rotting Oranges
- 01 Matrix
- As Far from Land as Possible
- Walls and Gates

5.4 State-Space Search

Examples:

- Open the Lock
- Sliding Puzzle
- Shortest Path Visiting All Nodes

5.5 Level-by-Level Expansion

Examples:

- Bus Routes
 - All Nodes Distance K in Binary Tree
 - shortest transformation sequence
-

6. When NOT to Use Graph BFS

6.1 Weighted Shortest Path

If edges have different positive weights, use Dijkstra.

If negative weights exist, use Bellman-Ford.

BFS only works for equal edge costs.

6.2 Need All Paths

Use DFS/backtracking.

BFS can find shortest paths, but generating all paths is often more complex.

6.3 Need Connected Components Only

DFS or BFS both work.

DFS is often simpler.

6.4 Need Topological Order

Use Kahn's BFS indegree algorithm or DFS topological sort.

This is a special BFS variant covered later.

6.5 Huge State Space Without Pruning

BFS can explode in memory because it stores a whole frontier.

7. Core Intuition

BFS works in waves.

Start from one or more sources.

Then expand to all neighbors.

Then expand to neighbors of neighbors.

Each layer represents one more step.

step 0: start

step 1: all nodes one edge away

step 2: all nodes two edges away

step 3: all nodes three edges away

The first time you reach the target, the step count is minimum.

8. Mental Models

8.1 Fire Spreading

Fire spreads from a starting cell to adjacent cells every minute.
This models Rotting Oranges.

8.2 Ripples in Water

A stone dropped in water creates expanding rings.
BFS expands like rings.

8.3 Queue of Frontiers

The queue stores nodes discovered but not processed yet.
FIFO order ensures closer nodes are processed before farther nodes.

8.4 First Visit Is Best Visit

In unweighted BFS:

first time reaching a node = shortest distance to that node

So mark visited when pushing into queue.

9. Pattern Recognition Checklist

Ask:

19. Does the problem ask for minimum steps?
20. Does each move have equal cost?
21. Is this a grid shortest path?
22. Is this a transformation problem?
23. Is this a spreading process?
24. Are there multiple starting sources?
25. Do I need nearest target?
26. Can I represent each state as a node?
27. Can I generate neighbors from a state?
28. Do I need level count?

If yes, Graph BFS is likely useful.

10. C++ Tools Needed

10.1 Queue

```
queue<int> q;
```

For grid:

```
queue<pair<int, int>> q;
```

For state BFS:

```
queue<string> q;
```

For complex state:

```
queue<tuple<int, int, int>> q;
```

10.2 Visited Array

```
vector<bool> visited(n, false);
```

10.3 Visited Set

For strings or complex states:

```
unordered_set<string> visited;
```

10.4 Distance Array

```
vector<int> dist(n, -1);
```

10.5 Grid Directions

4-direction:

```
vector<pair<int, int>> dirs = {  
    {1, 0},  
    {-1, 0},  
    {0, 1},  
    {0, -1}  
};
```

8-direction:

```
vector<pair<int, int>> dirs = {  
    {1, 0}, {-1, 0}, {0, 1}, {0, -1},  
    {1, 1}, {1, -1}, {-1, 1}, {-1, -1}  
};
```

11. Core Templates

Template 1 — Basic BFS

```
void bfs(int start, vector<vector<int>>& graph) {  
    vector<bool> visited(graph.size(), false);  
    queue<int> q;  
  
    q.push(start);  
    visited[start] = true;  
  
    while (!q.empty()) {  
        int node = q.front();  
        q.pop();  
  
        for (int neighbor : graph[node]) {  
            if (!visited[neighbor]) {  
                visited[neighbor] = true;  
                q.push(neighbor);  
            }  
        }  
    }  
}
```

```
}  
}
```

Template 2 — BFS Shortest Distance

```
vector<int> shortestDistance(int start, vector<vector<int>>& graph) {  
    int n = graph.size();  
  
    vector<int> dist(n, -1);  
    queue<int> q;  
  
    q.push(start);  
    dist[start] = 0;  
  
    while (!q.empty()) {  
        int node = q.front();  
        q.pop();  
  
        for (int neighbor : graph[node]) {  
            if (dist[neighbor] == -1) {  
                dist[neighbor] = dist[node] + 1;  
                q.push(neighbor);  
            }  
        }  
    }  
  
    return dist;  
}
```

Template 3 — BFS With Level Size

```
int steps = 0;  
  
queue<int> q;  
q.push(start);  
visited[start] = true;  
  
while (!q.empty()) {  
    int levelSize = q.size();  
  
    for (int i = 0; i < levelSize; i++) {  
        int node = q.front();  
        q.pop();
```

```

        if (node == target) {
            return steps;
        }

        for (int neighbor : graph[node]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }

    steps++;
}

```

Template 4 — Grid BFS

```

int bfs(vector<vector<int>>& grid, int sr, int sc) {
    int rows = grid.size();
    int cols = grid[0].size();

    queue<pair<int, int>> q;
    q.push({sr, sc});

    vector<vector<bool>> visited(rows, vector<bool>(cols, false));
    visited[sr][sc] = true;

    vector<pair<int, int>> dirs = {
        {1, 0},
        {-1, 0},
        {0, 1},
        {0, -1}
    };

    int steps = 0;

    while (!q.empty()) {
        int levelSize = q.size();

        for (int i = 0; i < levelSize; i++) {
            auto [r, c] = q.front();
            q.pop();

```

```

    for (auto [dr, dc] : dirs) {
        int nr = r + dr;
        int nc = c + dc;

        if (nr < 0 || nc < 0 || nr >= rows || nc >= cols) {
            continue;
        }

        if (visited[nr][nc]) {
            continue;
        }

        visited[nr][nc] = true;
        q.push({nr, nc});
    }
}

steps++;
}

return steps;
}

```

Template 5 — Multi-Source BFS

```

queue<pair<int, int>> q;
vector<vector<int>> dist(rows, vector<int>(cols, -1));

for (int r = 0; r < rows; r++) {
    for (int c = 0; c < cols; c++) {
        if (isSource(r, c)) {
            q.push({r, c});
            dist[r][c] = 0;
        }
    }
}

while (!q.empty()) {
    auto [r, c] = q.front();
    q.pop();

    for (auto [dr, dc] : dirs) {

```

```
int nr = r + dr;
int nc = c + dc;

if (invalid) {
    continue;
}

if (dist[nr][nc] != -1) {
    continue;
}

dist[nr][nc] = dist[r][c] + 1;
q.push({nr, nc});
}
}
```

Template 6 — State BFS

```
int bfs(string start, string target) {
    queue<string> q;
    unordered_set<string> visited;

    q.push(start);
    visited.insert(start);

    int steps = 0;

    while (!q.empty()) {
        int levelSize = q.size();

        for (int i = 0; i < levelSize; i++) {
            string state = q.front();
            q.pop();

            if (state == target) {
                return steps;
            }

            for (string nextState : getNeighbors(state)) {
                if (!visited.count(nextState)) {
                    visited.insert(nextState);
                    q.push(nextState);
                }
            }
        }
    }
}
```

```

    }
}

steps++;
}

return -1;
}

```

12. Major BFS Variations

Standard BFS	visit reachable nodes	Valid Path
Shortest path BFS	first target reach is shortest	Binary Matrix
Multi-source BFS	start from all sources	Rotting Oranges
Grid BFS	cells as nodes	Nearest Exit
State BFS	states as nodes	Open Lock
Word BFS	words as nodes	Word Ladder
BFS with metadata	store extra state	Obstacle Elimination
BFS with bitmask	visited subset state	Shortest Path Visiting All Nodes
Bidirectional BFS	expand from both ends	Word Ladder optimization

13. Full Problem Walkthrough 1 — LeetCode 1971: Find if Path Exists in Graph

13.1 Problem Summary

Given undirected graph, source, and destination.
Return true if there is a path from source to destination.

13.2 How to Identify BFS

Reachability can be solved with either DFS or BFS.
Use BFS from source.

13.3 C++ Code

```
class Solution {
public:
    bool validPath(int n,
                  vector<vector<int>>& edges,
                  int source,
                  int destination) {
        vector<vector<int>> graph(n);

        for (auto& edge : edges) {
            int u = edge[0];
            int v = edge[1];

            graph[u].push_back(v);
            graph[v].push_back(u);
        }

        vector<bool> visited(n, false);
        queue<int> q;

        q.push(source);
        visited[source] = true;

        while (!q.empty()) {
            int node = q.front();
            q.pop();

            if (node == destination) {
                return true;
            }

            for (int neighbor : graph[node]) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    q.push(neighbor);
                }
            }
        }

        return false;
    }
};
```

```
}  
};
```

13.4 Complexity

Time:

$O(V + E)$

Space:

$O(V + E)$

14. Full Problem Walkthrough 2 — LeetCode 1091: Shortest Path in Binary Matrix

14.1 Problem Summary

Given $n \times n$ binary matrix.

You can move in 8 directions.

0 means open cell.

1 means blocked cell.

Return shortest path length from top-left to bottom-right.

14.2 How to Identify BFS

Need shortest path in an unweighted grid.

Use BFS.

14.3 C++ Code

```
class Solution {  
public:  
    int shortestPathBinaryMatrix(vector<vector<int>>& grid) {  
        int n = grid.size();  
  
        if (grid[0][0] == 1 || grid[n - 1][n - 1] == 1) {  
            return -1;  
        }  
  
        vector<pair<int, int>> dirs = {  
            {1, 0}, {-1, 0}, {0, 1}, {0, -1},  
            {1, 1}, {1, -1}, {-1, 1}, {-1, -1}  
        };  
    };  
};
```

```

queue<pair<int, int>> q;
q.push({0, 0});

grid[0][0] = 1;

int pathLength = 1;

while (!q.empty()) {
    int levelSize = q.size();

    for (int i = 0; i < levelSize; i++) {
        auto [r, c] = q.front();
        q.pop();

        if (r == n - 1 && c == n - 1) {
            return pathLength;
        }

        for (auto [dr, dc] : dirs) {
            int nr = r + dr;
            int nc = c + dc;

            if (nr < 0 || nc < 0 || nr >= n || nc >= n) {
                continue;
            }

            if (grid[nr][nc] != 0) {
                continue;
            }

            grid[nr][nc] = 1;
            q.push({nr, nc});
        }
    }

    pathLength++;
}

return -1;
}
};

```

14.4 Complexity

Time:

$O(n^2)$

Space:

$O(n^2)$

14.5 Common Mistakes

- Forgetting 8 directions
- Returning edge count instead of path length
- Not checking blocked start/end
- Marking visited too late

15. Full Problem Walkthrough 3 — LeetCode 994: Rotting Oranges

15.1 Problem Summary

Grid contains:

0 = empty
1 = fresh orange
2 = rotten orange

Every minute, rotten oranges rot adjacent fresh oranges.
Return minimum minutes until no fresh oranges remain.
If impossible, return -1.

15.2 How to Identify BFS

Rot spreads level by level from all initially rotten oranges.
This is multi-source BFS.

15.3 C++ Code

```
class Solution {
public:
    int orangesRotting(vector<vector<int>>& grid) {
        int rows = grid.size();
        int cols = grid[0].size();

        queue<pair<int, int>> q;
        int fresh = 0;

        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
```

```

        if (grid[r][c] == 2) {
            q.push({r, c});
        } else if (grid[r][c] == 1) {
            fresh++;
        }
    }
}

if (fresh == 0) {
    return 0;
}

vector<pair<int, int>> dirs = {
    {1, 0},
    {-1, 0},
    {0, 1},
    {0, -1}
};

int minutes = 0;

while (!q.empty()) {
    int levelSize = q.size();
    bool rottedThisMinute = false;

    for (int i = 0; i < levelSize; i++) {
        auto [r, c] = q.front();
        q.pop();

        for (auto [dr, dc] : dirs) {
            int nr = r + dr;
            int nc = c + dc;

            if (nr < 0 || nc < 0 || nr >= rows || nc >= cols) {
                continue;
            }

            if (grid[nr][nc] != 1) {
                continue;
            }

            grid[nr][nc] = 2;

```

```

        fresh--;
        rottedThisMinute = true;
        q.push({nr, nc});
    }
}

if (rottedThisMinute) {
    minutes++;
}
}

return fresh == 0 ? minutes : -1;
}
};

```

15.4 Complexity

Time:

$O(\text{rows} * \text{cols})$

Space:

$O(\text{rows} * \text{cols})$

15.5 Common Mistakes

- Starting BFS from only one rotten orange
- Incrementing minutes one extra time
- Not counting remaining fresh oranges
- Not using multi-source BFS

16. Full Problem Walkthrough 4 — Walls and Gates

16.1 Problem Summary

Given grid with:

INF = empty room

-1 = wall

0 = gate

Fill each empty room with distance to nearest gate.

If impossible, leave as INF.

16.2 How to Identify BFS

Need nearest gate for every room.

Start BFS from all gates at once.

This is multi-source BFS.

16.3 C++ Code

```
void wallsAndGates(vector<vector<int>>& rooms) {
    int rows = rooms.size();
    int cols = rooms[0].size();

    queue<pair<int, int>> q;

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (rooms[r][c] == 0) {
                q.push({r, c});
            }
        }
    }

    vector<pair<int, int>> dirs = {
        {1, 0},
        {-1, 0},
        {0, 1},
        {0, -1}
    };

    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();

        for (auto [dr, dc] : dirs) {
            int nr = r + dr;
            int nc = c + dc;

            if (nr < 0 || nc < 0 || nr >= rows || nc >= cols) {
                continue;
            }

            if (rooms[nr][nc] != INT_MAX) {
                continue;
            }

            rooms[nr][nc] = rooms[r][c] + 1;
            q.push({nr, nc});
        }
    }
}
```

```
}  
}  
}
```

16.4 Complexity

Time:

$O(\text{rows} * \text{cols})$

Space:

$O(\text{rows} * \text{cols})$

17. Full Problem Walkthrough 5 — LeetCode 752: Open the Lock

17.1 Problem Summary

You have a lock with 4 wheels.

Each wheel has digits 0..9.

Start:

"0000"

Target is given.

Deadends cannot be used.

Each move turns one wheel up or down by 1.

Return minimum moves to target.

17.2 How to Identify BFS

Each lock combination is a state.

Each move has equal cost.

Need minimum moves.

Use state-space BFS.

17.3 Neighbor Generation

For each of 4 positions:

- turn digit up
- turn digit down

17.4 C++ Code

```
class Solution {  
public:  
    vector<string> getNeighbors(string state) {  
        vector<string> neighbors;  
  
        for (int i = 0; i < 4; i++) {
```

```

    string up = state;
    string down = state;

    up[i] = up[i] == '9' ? '0' : up[i] + 1;
    down[i] = down[i] == '0' ? '9' : down[i] - 1;

    neighbors.push_back(up);
    neighbors.push_back(down);
}

return neighbors;
}

int openLock(vector<string>& deadends, string target) {
    unordered_set<string> dead(deadends.begin(), deadends.end());

    if (dead.count("0000")) {
        return -1;
    }

    queue<string> q;
    unordered_set<string> visited;

    q.push("0000");
    visited.insert("0000");

    int moves = 0;

    while (!q.empty()) {
        int levelSize = q.size();

        for (int i = 0; i < levelSize; i++) {
            string state = q.front();
            q.pop();

            if (state == target) {
                return moves;
            }

            for (string nextState : getNeighbors(state)) {
                if (dead.count(nextState) || visited.count(nextState)) {
                    continue;
                }
            }
        }
    }
}

```

```

        }

        visited.insert(nextState);
        q.push(nextState);
    }
}

moves++;
}

return -1;
}
};

```

17.5 Complexity

There are at most:

10000

states.

Each state generates 8 neighbors.

Time:

$O(10000)$

Space:

$O(10000)$

18. Full Problem Walkthrough 6 — LeetCode 127: Word Ladder

18.1 Problem Summary

Given beginWord, endWord, and word list.

Change one letter at a time.

Each intermediate word must be in word list.

Return length of shortest transformation sequence.

18.2 How to Identify BFS

Words are states.

Edges connect words differing by one character.

Need shortest transformation length.

Use BFS.

18.3 Simple Neighbor Generation Code

```

class Solution {
public:
    int ladderLength(string beginWord,
                    string endWord,
                    vector<string>& wordList) {
        unordered_set<string> dict(wordList.begin(), wordList.end());

        if (!dict.count(endWord)) {
            return 0;
        }

        queue<string> q;
        q.push(beginWord);

        unordered_set<string> visited;
        visited.insert(beginWord);

        int length = 1;

        while (!q.empty()) {
            int levelSize = q.size();

            for (int i = 0; i < levelSize; i++) {
                string word = q.front();
                q.pop();

                if (word == endWord) {
                    return length;
                }

                for (int pos = 0; pos < word.size(); pos++) {
                    char original = word[pos];

                    for (char c = 'a'; c <= 'z'; c++) {
                        if (c == original) {
                            continue;
                        }

                        word[pos] = c;

                        if (dict.count(word) && !visited.count(word)) {
                            visited.insert(word);
                        }
                    }
                }
            }
        }
    }
};

```

```

        q.push(word);
    }
}

word[pos] = original;
}
}

length++;
}

return 0;
}
};

```

18.4 Complexity

Let:

N = number of words

L = word length

Each word generates:

$26 * L$

transformations.

Time roughly:

$O(N * L * 26)$

with hash lookup.

Space:

$O(N)$

18.5 Common Mistakes

- Returning number of edges instead of sequence length
- Not checking endWord exists
- Not restoring original character
- Not marking visited when pushing

19. Full Problem Walkthrough 7 — LeetCode 433: Minimum Genetic Mutation

19.1 Problem Summary

Given start gene, end gene, and bank.
Each mutation changes one character.
Valid characters:

A, C, G, T

Each mutation must exist in bank.
Return minimum mutations.

19.2 How to Identify BFS

Same as Word Ladder, but alphabet has 4 choices.
Need minimum transformations.

19.3 C++ Code

```
class Solution {
public:
    int minMutation(string startGene,
                    string endGene,
                    vector<string>& bank) {
        unordered_set<string> valid(bank.begin(), bank.end());

        if (!valid.count(endGene)) {
            return -1;
        }

        vector<char> chars = {'A', 'C', 'G', 'T'};

        queue<string> q;
        unordered_set<string> visited;

        q.push(startGene);
        visited.insert(startGene);

        int steps = 0;

        while (!q.empty()) {
            int levelSize = q.size();

            for (int i = 0; i < levelSize; i++) {
                string gene = q.front();
                q.pop();

                if (gene == endGene) {
                    return steps;
                }
            }
            steps++;
        }
    }
};
```

```

    }

    for (int pos = 0; pos < gene.size(); pos++) {
        char original = gene[pos];

        for (char c : chars) {
            if (c == original) {
                continue;
            }

            gene[pos] = c;

            if (valid.count(gene) && !visited.count(gene)) {
                visited.insert(gene);
                q.push(gene);
            }
        }

        gene[pos] = original;
    }

    steps++;
}

return -1;
}
};

```

19.4 Complexity

Let N be bank size and gene length be 8.

Time:

O(N)

up to a small constant neighbor generation cost.

Space:

O(N)

20. Full Problem Walkthrough 8 — LeetCode 1926: Nearest Exit from Entrance in Maze

20.1 Problem Summary

Maze contains:

'.' = empty
'+' = wall

Given entrance cell.
Return minimum steps to nearest exit.
Exit is empty cell on border, excluding entrance.

20.2 How to Identify BFS

Need nearest exit in unweighted grid.
Use BFS from entrance.

20.3 C++ Code

```
class Solution {
public:
    int nearestExit(vector<vector<char>>& maze, vector<int>& entrance) {
        int rows = maze.size();
        int cols = maze[0].size();

        queue<pair<int, int>> q;
        q.push({entrance[0], entrance[1]});

        maze[entrance[0]][entrance[1]] = '+';

        vector<pair<int, int>> dirs = {
            {1, 0},
            {-1, 0},
            {0, 1},
            {0, -1}
        };

        int steps = 0;

        while (!q.empty()) {
            int levelSize = q.size();
```

```

for (int i = 0; i < levelSize; i++) {
    auto [r, c] = q.front();
    q.pop();

    if (steps > 0 &&
        (r == 0 || c == 0 || r == rows - 1 || c == cols - 1)) {
        return steps;
    }

    for (auto [dr, dc] : dirs) {
        int nr = r + dr;
        int nc = c + dc;

        if (nr < 0 || nc < 0 || nr >= rows || nc >= cols) {
            continue;
        }

        if (maze[nr][nc] != '.') {
            continue;
        }

        maze[nr][nc] = '+';
        q.push({nr, nc});
    }

    steps++;
}

return -1;
}
};

```

20.4 Complexity

Time:

$O(\text{rows} * \text{cols})$

Space:

$O(\text{rows} * \text{cols})$

20.5 Common Mistake

Counting entrance as an exit.

The entrance is not an exit even if it is on border.

21. Full Problem Walkthrough 9 — LeetCode 909: Snakes and Ladders

21.1 Problem Summary

Given board with labels from 1 to n^2 .

Each move, choose next square from current +1 to current +6.

If square has snake or ladder, move to destination.

Return minimum moves to reach n^2 .

21.2 How to Identify BFS

Each square is a node.

Dice roll creates edges.

Each move has equal cost.

Need minimum moves.

21.3 Coordinate Conversion

Board labels zigzag from bottom row.

Need convert square number to row/col.

21.4 C++ Code

```
class Solution {
public:
    pair<int, int> getPosition(int square, int n) {
        int quot = (square - 1) / n;
        int rem = (square - 1) % n;

        int row = n - 1 - quot;
        int col;

        if (quot % 2 == 0) {
            col = rem;
        } else {
            col = n - 1 - rem;
        }

        return {row, col};
    }

    int snakesAndLadders(vector<vector<int>>& board) {
        int n = board.size();
```

```

int target = n * n;

queue<int> q;
vector<bool> visited(target + 1, false);

q.push(1);
visited[1] = true;

int moves = 0;

while (!q.empty()) {
    int levelSize = q.size();

    for (int i = 0; i < levelSize; i++) {
        int square = q.front();
        q.pop();

        if (square == target) {
            return moves;
        }

        for (int next = square + 1; next <= min(square + 6, target); next++) {
            auto [r, c] = getPosition(next, n);

            int destination = board[r][c] == -1 ? next : board[r][c];

            if (!visited[destination]) {
                visited[destination] = true;
                q.push(destination);
            }
        }
    }

    moves++;
}

return -1;
}
};

```

21.5 Complexity

There are n^2 squares.

Each has at most 6 moves.

Time:

$O(n^2)$

Space:

$O(n^2)$

22. Full Problem Walkthrough 10 — LeetCode 542: 01 Matrix

22.1 Problem Summary

Given binary matrix, return distance of each cell to nearest 0.

22.2 How to Identify BFS

Need nearest 0 for all cells.

Start BFS from all 0 cells.

This is multi-source BFS.

22.3 C++ Code

```
class Solution {
public:
    vector<vector<int>> updateMatrix(vector<vector<int>>& mat) {
        int rows = mat.size();
        int cols = mat[0].size();

        queue<pair<int, int>> q;
        vector<vector<int>> dist(rows, vector<int>(cols, -1));

        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                if (mat[r][c] == 0) {
                    q.push({r, c});
                    dist[r][c] = 0;
                }
            }
        }

        vector<pair<int, int>> dirs = {
            {1, 0},
            {-1, 0},
            {0, 1},
            {0, -1}
        }
```

```

};

while (!q.empty()) {
    auto [r, c] = q.front();
    q.pop();

    for (auto [dr, dc] : dirs) {
        int nr = r + dr;
        int nc = c + dc;

        if (nr < 0 || nc < 0 || nr >= rows || nc >= cols) {
            continue;
        }

        if (dist[nr][nc] != -1) {
            continue;
        }

        dist[nr][nc] = dist[r][c] + 1;
        q.push({nr, nc});
    }
}

return dist;
}
};

```

22.4 Complexity

Time:

$O(\text{rows} * \text{cols})$

Space:

$O(\text{rows} * \text{cols})$

23. Full Problem Walkthrough 11 — LeetCode 1162: As Far from Land as Possible

23.1 Problem Summary

Given grid:

1 = land
0 = water

Return maximum distance from water cell to nearest land.
If grid has only land or only water, return -1.

23.2 How to Identify BFS

Need distance from each water cell to nearest land.
Start BFS from all land cells.

23.3 C++ Code

```
class Solution {
public:
    int maxDistance(vector<vector<int>>& grid) {
        int n = grid.size();

        queue<pair<int, int>> q;
        int land = 0;
        int water = 0;

        for (int r = 0; r < n; r++) {
            for (int c = 0; c < n; c++) {
                if (grid[r][c] == 1) {
                    q.push({r, c});
                    land++;
                } else {
                    water++;
                }
            }
        }

        if (land == 0 || water == 0) {
            return -1;
        }

        vector<pair<int, int>> dirs = {
            {1, 0},
            {-1, 0},
            {0, 1},
            {0, -1}
        };

        int distance = -1;
```

```

while (!q.empty()) {
    int levelSize = q.size();
    distance++;

    for (int i = 0; i < levelSize; i++) {
        auto [r, c] = q.front();
        q.pop();

        for (auto [dr, dc] : dirs) {
            int nr = r + dr;
            int nc = c + dc;

            if (nr < 0 || nc < 0 || nr >= n || nc >= n) {
                continue;
            }

            if (grid[nr][nc] != 0) {
                continue;
            }

            grid[nr][nc] = 1;
            q.push({nr, nc});
        }
    }
}

return distance;
}
};

```

23.4 Complexity

Time:

$O(n^2)$

Space:

$O(n^2)$

24. Full Problem Walkthrough 12 — LeetCode 934: Shortest Bridge

24.1 Problem Summary

Grid has exactly two islands.

You can flip water 0 to land 1.

Return minimum number of flips to connect the two islands.

24.2 How to Identify BFS

Use DFS to mark first island.

Then BFS outward from first island until second island is reached.

This is DFS + multi-source BFS.

24.3 C++ Code

```
class Solution {
public:
    vector<pair<int, int>> dirs = {
        {1, 0},
        {-1, 0},
        {0, 1},
        {0, -1}
    };

    void dfs(vector<vector<int>>& grid,
            int r,
            int c,
            queue<pair<int, int>>& q) {
        int n = grid.size();

        if (r < 0 || c < 0 || r >= n || c >= n) {
            return;
        }

        if (grid[r][c] != 1) {
            return;
        }

        grid[r][c] = 2;
        q.push({r, c});

        for (auto [dr, dc] : dirs) {
            dfs(grid, r + dr, c + dc, q);
        }
    }
}
```

```

int shortestBridge(vector<vector<int>>& grid) {
    int n = grid.size();
    queue<pair<int, int>> q;

    bool found = false;

    for (int r = 0; r < n && !found; r++) {
        for (int c = 0; c < n && !found; c++) {
            if (grid[r][c] == 1) {
                dfs(grid, r, c, q);
                found = true;
            }
        }
    }

    int flips = 0;

    while (!q.empty()) {
        int levelSize = q.size();

        for (int i = 0; i < levelSize; i++) {
            auto [r, c] = q.front();
            q.pop();

            for (auto [dr, dc] : dirs) {
                int nr = r + dr;
                int nc = c + dc;

                if (nr < 0 || nc < 0 || nr >= n || nc >= n) {
                    continue;
                }

                if (grid[nr][nc] == 1) {
                    return flips;
                }

                if (grid[nr][nc] == 0) {
                    grid[nr][nc] = 2;
                    q.push({nr, nc});
                }
            }
        }
        flips++;
    }
}

```

```
        flips++;  
    }  
  
    return -1;  
}  
};
```

24.4 Complexity

Time:

$O(n^2)$

Space:

$O(n^2)$

25. Full Problem Walkthrough 13 — LeetCode 815: Bus Routes

25.1 Problem Summary

Given bus routes.

Each route contains stops.

You can board buses and transfer at stops.

Return minimum number of buses needed from source to target.

25.2 How to Identify BFS

Need minimum number of buses.

A BFS level can represent taking one more bus.

We need map:

stop -> routes containing this stop

25.3 C++ Code

```
class Solution {  
public:  
    int numBusesToDestination(vector<vector<int>>& routes,  
                             int source,  
                             int target) {  
        if (source == target) {  
            return 0;  
        }  
  
        unordered_map<int, vector<int>> stopToRoutes;
```

```

for (int i = 0; i < routes.size(); i++) {
    for (int stop : routes[i]) {
        stopToRoutes[stop].push_back(i);
    }
}

queue<int> q;
unordered_set<int> visitedStops;
vector<bool> visitedRoutes(routes.size(), false);

q.push(source);
visitedStops.insert(source);

int buses = 0;

while (!q.empty()) {
    int levelSize = q.size();
    buses++;

    for (int i = 0; i < levelSize; i++) {
        int stop = q.front();
        q.pop();

        for (int routeIndex : stopToRoutes[stop]) {
            if (visitedRoutes[routeIndex]) {
                continue;
            }

            visitedRoutes[routeIndex] = true;

            for (int nextStop : routes[routeIndex]) {
                if (nextStop == target) {
                    return buses;
                }

                if (!visitedStops.count(nextStop)) {
                    visitedStops.insert(nextStop);
                    q.push(nextStop);
                }
            }
        }
    }
}

```

```

    }
}

return -1;
}
};

```

25.4 Complexity

Let total route entries be S .

Time:

$O(S)$

Space:

$O(S)$

25.5 Common Mistakes

- Treating stops as moves instead of buses
- Not marking routes visited
- Revisiting same route many times

26. Full Problem Walkthrough 14 — All Nodes Distance K in Binary Tree

26.1 Problem Summary

Given binary tree, target node, and integer k .

Return all node values distance k from target.

26.2 How to Identify BFS

A binary tree normally has downward edges only.

But distance can go up to parent too.

Build parent map, then BFS from target as if graph is undirected.

26.3 C++ Code

```

class Solution {
public:
    void buildParent(TreeNode* root,
                    TreeNode* parent,
                    unordered_map<TreeNode*, TreeNode*>& parentMap) {
        if (root == nullptr) {
            return;
        }
    }
}

```

```

parentMap[root] = parent;

buildParent(root->left, root, parentMap);
buildParent(root->right, root, parentMap);
}

vector<int> distanceK(TreeNode* root, TreeNode* target, int k) {
    unordered_map<TreeNode*, TreeNode*> parentMap;
    buildParent(root, nullptr, parentMap);

    unordered_set<TreeNode*> visited;
    queue<TreeNode*> q;

    q.push(target);
    visited.insert(target);

    int distance = 0;

    while (!q.empty()) {
        int levelSize = q.size();

        if (distance == k) {
            vector<int> result;

            while (!q.empty()) {
                result.push_back(q.front()->val);
                q.pop();
            }

            return result;
        }

        for (int i = 0; i < levelSize; i++) {
            TreeNode* node = q.front();
            q.pop();

            vector<TreeNode*> neighbors = {
                node->left,
                node->right,
                parentMap[node]
            };

```

```

        for (TreeNode* neighbor : neighbors) {
            if (neighbor != nullptr && !visited.count(neighbor)) {
                visited.insert(neighbor);
                q.push(neighbor);
            }
        }
    }

    distance++;
}

return {};
}
};

```

26.4 Complexity

Time:

$O(n)$

Space:

$O(n)$

27. Full Problem Walkthrough 15 — Minimum Knight Moves

27.1 Problem Summary

Given a chess knight at (0,0), find minimum moves to reach (x,y).

Knight has 8 possible moves.

27.2 How to Identify BFS

Each board position is a state.

Each knight move has equal cost.

Need minimum moves.

27.3 C++ Code

```

int minKnightMoves(int x, int y) {
    x = abs(x);
    y = abs(y);

    vector<pair<int, int>> dirs = {

```

```

    {1, 2}, {2, 1}, {-1, 2}, {-2, 1},
    {1, -2}, {2, -1}, {-1, -2}, {-2, -1}
};

queue<pair<int, int>> q;
set<pair<int, int>> visited;

q.push({0, 0});
visited.insert({0, 0});

int steps = 0;

while (!q.empty()) {
    int levelSize = q.size();

    for (int i = 0; i < levelSize; i++) {
        auto [r, c] = q.front();
        q.pop();

        if (r == x && c == y) {
            return steps;
        }

        for (auto [dr, dc] : dirs) {
            int nr = r + dr;
            int nc = c + dc;

            if (nr < -2 || nc < -2 || nr > x + 2 || nc > y + 2) {
                continue;
            }

            if (!visited.count({nr, nc})) {
                visited.insert({nr, nc});
                q.push({nr, nc});
            }
        }
    }

    steps++;
}

return -1;

```

```
}
```

27.4 Complexity

BFS explores bounded area around target.

Time and space depend on target distance.

28. Full Problem Walkthrough 16 — LeetCode 1129: Shortest Path with Alternating Colors

28.1 Problem Summary

Graph has red edges and blue edges.

Find shortest path from node 0 to every node such that edge colors alternate.

28.2 How to Identify BFS With State

The state is not just:

node

It is:

node + last edge color

Need visited by both node and color.

28.3 C++ Code

```
class Solution {
public:
    vector<int> shortestAlternatingPaths(int n,
                                       vector<vector<int>>& redEdges,
                                       vector<vector<int>>& blueEdges) {
        vector<vector<pair<int, int>>> graph(n);

        for (auto& edge : redEdges) {
            graph[edge[0]].push_back({edge[1], 0});
        }

        for (auto& edge : blueEdges) {
            graph[edge[0]].push_back({edge[1], 1});
        }

        vector<int> answer(n, -1);
        vector<vector<bool>> visited(n, vector<bool>(2, false));
```

```

queue<pair<int, int>> q;

q.push({0, 0});
q.push({0, 1});

visited[0][0] = true;
visited[0][1] = true;

int distance = 0;

while (!q.empty()) {
    int levelSize = q.size();

    for (int i = 0; i < levelSize; i++) {
        auto [node, lastColor] = q.front();
        q.pop();

        if (answer[node] == -1) {
            answer[node] = distance;
        }

        for (auto [neighbor, edgeColor] : graph[node]) {
            if (edgeColor == lastColor) {
                continue;
            }

            if (visited[neighbor][edgeColor]) {
                continue;
            }

            visited[neighbor][edgeColor] = true;
            q.push({neighbor, edgeColor});
        }
    }

    distance++;
}

return answer;
}
};

```

28.4 Complexity

Time:

$O(V + E)$

Space:

$O(V + E)$

29. Full Problem Walkthrough 17 — LeetCode 1293: Shortest Path in a Grid with Obstacles Elimination

29.1 Problem Summary

Grid contains:

0 = empty

1 = obstacle

You can eliminate at most k obstacles.

Return shortest path from top-left to bottom-right.

29.2 How to Identify BFS With State

State is:

row, col, remaining eliminations

Visiting same cell with more remaining eliminations is better than with fewer.

29.3 C++ Code

```
class Solution {
public:
    int shortestPath(vector<vector<int>>& grid, int k) {
        int rows = grid.size();
        int cols = grid[0].size();

        if (rows == 1 && cols == 1) {
            return 0;
        }

        vector<pair<int, int>> dirs = {
            {1, 0},
            {-1, 0},
            {0, 1},
```

```

    {0, -1}
};

queue<tuple<int, int, int>> q;
q.push({0, 0, k});

vector<vector<int>> best(rows, vector<int>(cols, -1));
best[0][0] = k;

int steps = 0;

while (!q.empty()) {
    int levelSize = q.size();

    for (int i = 0; i < levelSize; i++) {
        auto [r, c, remaining] = q.front();
        q.pop();

        if (r == rows - 1 && c == cols - 1) {
            return steps;
        }

        for (auto [dr, dc] : dirs) {
            int nr = r + dr;
            int nc = c + dc;

            if (nr < 0 || nc < 0 || nr >= rows || nc >= cols) {
                continue;
            }

            int nextRemaining = remaining - grid[nr][nc];

            if (nextRemaining < 0) {
                continue;
            }

            if (best[nr][nc] >= nextRemaining) {
                continue;
            }

            best[nr][nc] = nextRemaining;
            q.push({nr, nc, nextRemaining});
        }
    }
}

```

```

        }
    }

    steps++;
}

return -1;
}
};

```

29.4 Complexity

Time:

$O(\text{rows} * \text{cols} * k)$

Space:

$O(\text{rows} * \text{cols})$

with the optimized best matrix, though state count can still be up to $\text{rows} * \text{cols} * k$.

30. Full Problem Walkthrough 18 — LeetCode 847: Shortest Path Visiting All Nodes

30.1 Problem Summary

Given connected undirected graph.

Return shortest path length that visits every node at least once.

You may revisit nodes.

30.2 How to Identify BFS With Bitmask

State must include:

current node
visited node mask

Goal mask:

all nodes visited

This is BFS over state space.

30.3 C++ Code

```

class Solution {
public:
    int shortestPathLength(vector<vector<int>>& graph) {

```

```

int n = graph.size();

if (n == 1) {
    return 0;
}

int targetMask = (1 << n) - 1;

queue<pair<int, int>> q;
vector<vector<bool>> visited(n, vector<bool>(1 << n, false));

for (int node = 0; node < n; node++) {
    int mask = 1 << node;

    q.push({node, mask});
    visited[node][mask] = true;
}

int steps = 0;

while (!q.empty()) {
    int levelSize = q.size();

    for (int i = 0; i < levelSize; i++) {
        auto [node, mask] = q.front();
        q.pop();

        if (mask == targetMask) {
            return steps;
        }

        for (int neighbor : graph[node]) {
            int nextMask = mask | (1 << neighbor);

            if (!visited[neighbor][nextMask]) {
                visited[neighbor][nextMask] = true;
                q.push({neighbor, nextMask});
            }
        }
    }

    steps++;
}

```

```
    }  
  
    return -1;  
}  
};
```

30.4 Complexity

States:

$n * 2^n$

Time:

$O((V + E) * 2^V)$

Space:

$O(V * 2^V)$

31. Mini Problem 1 — BFS Distance From Source

```
vector<int> bfsDistance(int n, vector<vector<int>>& graph, int source) {  
    vector<int> dist(n, -1);  
    queue<int> q;  
  
    q.push(source);  
    dist[source] = 0;  
  
    while (!q.empty()) {  
        int node = q.front();  
        q.pop();  
  
        for (int neighbor : graph[node]) {  
            if (dist[neighbor] == -1) {  
                dist[neighbor] = dist[node] + 1;  
                q.push(neighbor);  
            }  
        }  
    }  
  
    return dist;  
}
```

32. Mini Problem 2 — Multi-Source BFS Distance Grid

```
vector<vector<int>> multiSourceDistance(vector<vector<int>>& grid) {
    int rows = grid.size();
    int cols = grid[0].size();

    queue<pair<int, int>> q;
    vector<vector<int>> dist(rows, vector<int>(cols, -1));

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == 1) {
                q.push({r, c});
                dist[r][c] = 0;
            }
        }
    }

    vector<pair<int, int>> dirs = {
        {1, 0},
        {-1, 0},
        {0, 1},
        {0, -1}
    };

    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();

        for (auto [dr, dc] : dirs) {
            int nr = r + dr;
            int nc = c + dc;

            if (nr < 0 || nc < 0 || nr >= rows || nc >= cols) {
                continue;
            }

            if (dist[nr][nc] != -1) {
                continue;
            }
        }
    }
}
```

```

        dist[nr][nc] = dist[r][c] + 1;
        q.push({nr, nc});
    }
}

return dist;
}

```

33. Mini Problem 3 — BFS With Parent Reconstruction

```

vector<int> shortestPath(int n, vector<vector<int>>& graph, int source, int
target) {
    vector<int> parent(n, -1);
    vector<bool> visited(n, false);

    queue<int> q;
    q.push(source);
    visited[source] = true;

    while (!q.empty()) {
        int node = q.front();
        q.pop();

        if (node == target) {
            break;
        }

        for (int neighbor : graph[node]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                parent[neighbor] = node;
                q.push(neighbor);
            }
        }
    }

    if (!visited[target]) {
        return {};
    }
}

```

```
vector<int> path;

for (int cur = target; cur != -1; cur = parent[cur]) {
    path.push_back(cur);
}

reverse(path.begin(), path.end());

return path;
}
```

34. Common Edge Cases for Graph BFS

34.1 Start Equals Target

Return 0 steps or 1 length depending on problem.

Examples:

- Open Lock: moves = 0
- Word Ladder: sequence length may be 1 if allowed
- Grid shortest path: path length can be 1

34.2 Blocked Start or End

Return -1.

Common in grid shortest path problems.

34.3 Empty Graph

Handle based on problem constraints.

34.4 Disconnected Graph

BFS may not reach target.

Return false, -1, or empty result.

34.5 Multiple Sources

Push all sources before BFS starts.

34.6 Marking Visited

Mark visited when pushing, not when popping, to avoid duplicate queue entries.

34.7 Level Count Off-by-One

Clarify whether answer counts:

- edges/moves
- nodes/path length
- minutes elapsed

35. Common Mistakes and Wrong Code

Mistake 1 — Using DFS for Shortest Path

DFS may find a path but not shortest.

Use BFS for unweighted shortest path.

Mistake 2 — Marking Visited Too Late

Wrong:

```
q.push(neighbor);
```

and mark after popping.

Correct:

```
visited[neighbor] = true;
```

```
q.push(neighbor);
```

Mistake 3 — Forgetting Multi-Source Setup

Rotting Oranges and 01 Matrix need all sources initially in queue.

Mistake 4 — Off-by-One Distance

Some problems count starting node.

Some count moves.

Check examples carefully.

Mistake 5 — Not Including State in Visited

For advanced BFS, visited may need:

```
node + color
```

```
row + col + remaining eliminations
```

```
node + bitmask
```

Not just node.

Mistake 6 — Reusing Grid Mutation Incorrectly

If grid values are meaningful later, use separate visited/distance matrix.

36. Interview Explanation Templates

36.1 Basic BFS

I use a queue to process nodes in the order they are discovered. I mark nodes visited when adding them to the queue to avoid cycles and repeated work.

36.2 Shortest Path

Because all edges have equal cost, BFS explores nodes in increasing distance. Therefore, the first time I reach the target, that distance is the shortest.

36.3 Grid BFS

I treat each cell as a graph node and use direction arrays to generate neighboring cells. BFS gives the minimum number of steps in the grid.

36.4 Multi-Source BFS

I push all starting sources into the queue first with distance 0. Then BFS expands from all sources simultaneously, so each cell gets distance to the nearest source.

36.5 State BFS

If reaching the same node with different conditions matters, I include those conditions in the BFS state and visited structure.

37. Complexity Patterns

Basic graph BFS	$O(V + E)$	$O(V + E)$
Shortest path unweighted	$O(V + E)$	$O(V)$
Grid BFS	$O(R * C)$	$O(R * C)$
Multi-source grid BFS	$O(R * C)$	$O(R * C)$
Open Lock	$O(10000)$	$O(10000)$
Word Ladder	$O(N * L * 26)$	$O(N)$
Rotting Oranges	$O(R * C)$	$O(R * C)$
01 Matrix	$O(R * C)$	$O(R * C)$
Bus Routes	$O(\text{total route entries})$	$O(\text{total route entries})$
Obstacle Elimination	$O(R * C * K)$	$O(R * C * K)$ conceptually
Shortest Path Visiting All Nodes	$O((V + E) * 2^V)$	$O(V * 2^V)$

38. Time-Boxed Practice Guidance

Easy Graph BFS

Target:

15-25 minutes

Examples:

- Find if Path Exists
- 01 Matrix
- Rotting Oranges
- Nearest Exit

Medium Graph BFS

Target:

25–45 minutes

Examples:

- Shortest Path in Binary Matrix
- Open the Lock
- Word Ladder
- Snakes and Ladders
- Shortest Bridge

Hard Graph BFS

Target:

45–75 minutes

Examples:

- Bus Routes
- Shortest Path with Obstacles Elimination
- Shortest Path Visiting All Nodes
- Sliding Puzzle

39. Expanded Practice Set

39.1 Must-Do Medium

29. Find if Path Exists in Graph
30. Shortest Path in Binary Matrix
31. Rotting Oranges
32. 01 Matrix
33. Open the Lock
34. Word Ladder
35. Minimum Genetic Mutation
36. Nearest Exit from Entrance in Maze
37. Snakes and Ladders
38. As Far from Land as Possible
39. Shortest Bridge

39.2 Strong Interview Practice

40. Bus Routes
41. All Nodes Distance K in Binary Tree
42. Shortest Path with Alternating Colors
43. Walls and Gates
44. Pacific Atlantic Water Flow, BFS version
45. Number of Enclaves
46. Map of Highest Peak
47. Minimum Knight Moves
48. Shortest Path in a Grid with Obstacles Elimination

39.3 Hard / Stretch

49. Shortest Path Visiting All Nodes
50. Sliding Puzzle
51. Word Ladder II
52. Cut Off Trees for Golf Event
53. Race Car

- 54. Minimum Moves to Move a Box to Their Target Location
 - 55. Escape a Large Maze
 - 56. Minimum Cost to Make at Least One Valid Path in a Grid, 0-1 BFS
-
- preview

40. Suggested Study Order

Step 1 — Basic BFS

Start with:

- 57. Find if Path Exists
- 58. BFS distance from source
- 59. Nearest Exit

Goal:

Understand queue, visited, and level count.

Step 2 — Grid Shortest Path

Solve:

- 60. Shortest Path in Binary Matrix
- 61. Rotting Oranges
- 62. 01 Matrix

Goal:

Master grid BFS and multi-source BFS.

Step 3 — State BFS

Solve:

- 63. Open the Lock
- 64. Minimum Genetic Mutation
- 65. Word Ladder

Goal:

Represent strings/states as graph nodes.

Step 4 — Board and Jump BFS

Solve:

- 66. Snakes and Ladders
- 67. Minimum Knight Moves

Goal:

Learn coordinate conversion and generated moves.

Step 5 — Advanced State BFS

Solve:

- 68. Shortest Path with Alternating Colors
- 69. Shortest Path with Obstacles Elimination
- 70. Shortest Path Visiting All Nodes

Goal:

Learn BFS where state includes extra metadata.

41. Pattern Decision Table

shortest path unweighted	standard BFS	Valid Path distance
grid minimum steps	grid BFS	Binary Matrix
spread/infection	multi-source BFS	Rotting Oranges
nearest source	multi-source BFS	01 Matrix
lock combinations	state BFS	Open Lock
word transformations	state BFS	Word Ladder
nearest exit	grid BFS	Maze Exit
dice board	BFS over squares	Snakes and Ladders
connect two islands	DFS mark + BFS expand	Shortest Bridge
minimum buses	BFS by bus layers	Bus Routes
tree distance with parent	graph BFS on tree	Distance K
alternating edge colors	node + color state	Alternating Colors
obstacle elimination	row + col + remaining k	Grid Obstacles
visit all nodes	node + bitmask	Shortest Path Visiting All Nodes

42. Final Cheat Sheet

Basic BFS

```
queue<int> q;  
q.push(start);  
visited[start] = true;  
  
while (!q.empty()) {  
    int node = q.front();  
    q.pop();
```

```

    for (int neighbor : graph[node]) {
        if (!visited[neighbor]) {
            visited[neighbor] = true;
            q.push(neighbor);
        }
    }
}

```

BFS Shortest Steps

```

int steps = 0;

while (!q.empty()) {
    int levelSize = q.size();

    for (int i = 0; i < levelSize; i++) {
        auto state = q.front();
        q.pop();

        if (state == target) {
            return steps;
        }

        // push next states
    }

    steps++;
}

```

Multi-Source BFS

```

for (all cells) {
    if (cell is source) {
        q.push(cell);
        dist[cell] = 0;
    }
}

```

Grid Directions

```

vector<pair<int, int>> dirs = {
    {1, 0},
    {-1, 0},
    {0, 1},
    {0, -1}
};

```

State BFS Visited

```
visited[row][col][remaining]
visited[node][color]
visited[node][mask]
```

Use enough state to distinguish meaningfully different situations.

43. What to Master Before Chapter 27

Before moving to Topological Sort, make sure you can:

- Explain BFS
- Use queue correctly
- Mark visited when pushing
- Count BFS levels
- Use BFS for shortest unweighted path
- Use BFS on grids
- Use multi-source BFS
- Generate state neighbors
- Solve Rotting Oranges
- Solve 01 Matrix
- Solve Open the Lock
- Solve Word Ladder
- Solve Shortest Path in Binary Matrix
- Recognize when BFS state needs extra metadata
- Avoid off-by-one distance errors

44. Core Takeaway

Graph BFS explores a graph level by level.

The most important fact is:

In an unweighted graph, BFS finds the shortest path.

Use BFS when the problem asks for:

- minimum steps
- nearest target
- shortest transformation
- spreading over time
- distance from multiple sources

The key question is:

What is one state, and how do I generate all next states?

Once you define the state and neighbors clearly, BFS usually becomes a queue-based template.